# Chapter 2

# Methodology

## 2.1 The LAS construction

LAS is a lattice adaptor signature whose base is a simplified, Dilithium-style Fiat–Shamir-with-aborts signature [9, 8]. All objects live in $R_q = \mathbb{Z}_q[X]/(X^d + 1)$, the public matrix has the form $A = [\, I_n \mid A' \,] \in R_q^{n \times (n+\ell)}$, and a key pair is a short secret $r$ with its image $t = A \cdot r$. The hard relation $(Y, y)$ reuses exactly that structure, so a statement is "just another public key": recovering its witness means finding a short preimage of $Y$ under $A$, a Module-SIS problem, while Module-LWE keeps the witness hidden [9]. *Key generation is shared* — LAS reuses the base KeyGen unchanged — so LAS is exactly the base signature plus four adaptor operations.

The base signature samples a masked commitment $w$, forms a challenge $c = H(pk, w, M)$, computes a response and rejects when $\|\mathbf{z}\|_\infty > \gamma - \kappa$. Relative to a statement $Y$, the adaptor extension [9] adds PreSign, which folds the statement into the hash, $c = H(pk, w + Y, M)$, and rejects at the *tighter* bound $\gamma - \kappa - 1$; PreVerify, which recomputes $w' = A\mathbf{z} - c\,t$ and checks $c = H(pk, w' + Y, M)$; Adapt, which adds the witness, $\mathbf{z} \leftarrow \hat{\mathbf{z}} + y$, producing a signature the base verifier accepts; and Extract, which recovers $y = \mathbf{z} - \hat{\mathbf{z}}$ (fig. 2.1).

$$\text{accept iff } \|\mathbf{z}\|_\infty \le \gamma - \kappa, \qquad \gamma = \kappa\, d\, (n + \ell). \tag{2.1}$$

The single subtle design point is the bound budget. The witness is ternary, so $\|y\|_\infty \le 1$, and because PreSign rejects at $\gamma - \kappa - 1$ the response after Adapt clears eq. (2.1); loosening it to $\gamma - \kappa$ would let adapted signatures exceed the

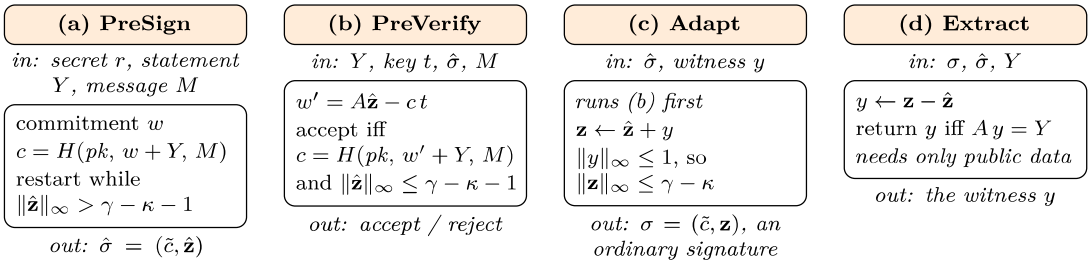


Figure 2.1: The four functions LAS adds to the base signature, one panel each: what goes in, what the function computes, and what comes out. KeyGen, Sign and Verify are reused unchanged and are not shown. Two asymmetries are the whole construction. Only (a) and (b) touch the statement $Y$, and only in one place — the challenge hash — which is why the adaptor layer inherits the base scheme's arithmetic rather than replacing it; only (c) and (d) touch the witness $y$, and they are the two functions a basic signature has no counterpart for. The bound in (a) is the one quantity that must not be relaxed: pre-signing rejects one unit tighter than eq. (2.1) precisely so that the addition in (c) cannot carry the adapted response past the bound the base verifier enforces. How the four compose, and why publishing $\sigma$ is what makes a swap atomic, is fig. 2.2.

bound and be rejected. Figure 2.2 shows the data flow.

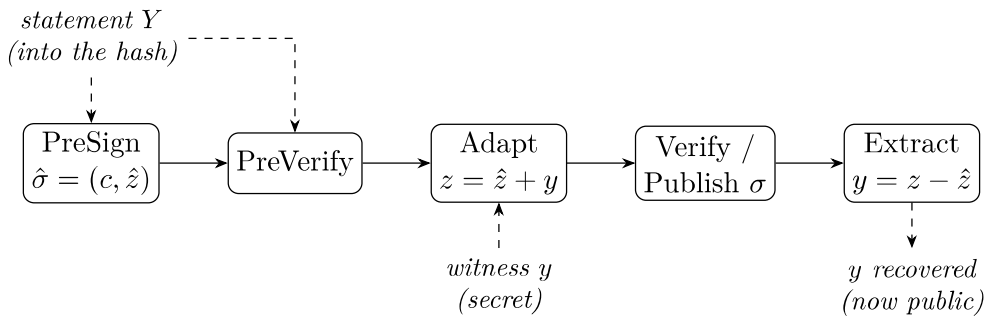


Figure 2.2: The LAS adaptor data flow. The statement $Y$ is folded into the challenge hash during PreSign and PreVerify; the witness $y$ is added by Adapt to turn the pre-signature into a base signature; and publishing that signature lets anyone holding the pre-signature run Extract to recover $y$. This last step is what makes an atomic swap atomic.

## 2.2 Implementation strategy: reuse, do not reinvent

The guiding principle is that an exotic scheme equals a basic scheme plus extra functions, so the lattice arithmetic should be *reused*, not rewritten. The implementation is built additively on the upstream CRYSTALS-Dilithium reference [8, 6] at a pinned commit, and the same strategy is applied again in Rust (section 2.5). Neither is itself "the reference": both are *modified, simplified* ML-DSA-style bases carrying the LAS layer of [9].

Classifying every source file by how the reference is used (table A.7) yields a clean split: the entire lattice core is reused unchanged, the only modified file is the `Makefile`, and the scheme, serialization, application and evaluation are new. No upstream function is modified, so the security-critical arithmetic is exactly the published reference code and every adaptor-specific line is isolated. The vendored `secp256k1-zkp` [4] and LaZer [14] are unmodified too, the latter behind a dedicated bridge.

**Data types and the matrix product.** LAS is a self-contained module over the reference's degree-$d$ polynomial type. Because $A = [\, I_n \mid A' \,]$, the product $A\,v = v_{\text{top}} + A' v_{\text{bot}}$ multiplies only $A'$.

**Hash, challenge, and samplers.** The random oracle $H$ is built from SHAKE256, absorbing a canonical encoding of the public key, then the commitment ($w$ for Sign, $w + Y$ for PreSign), then the message. The challenge sampler is the reference's `SampleInBall` at weight $\kappa$, giving $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Two samplers are added: a ternary one for secrets and witnesses, and a rejection sampler uniform on $[-\gamma, \gamma]$ for the mask (appendix A.3).

**Simplifications, and the modulus.** Following [9], the base signature omits Dilithium's hint vector, Power2Round and high/low-bit decomposition, hashing the full commitment $w$ rather than its high bits. This keeps the exact identity $A\mathbf{z} - c\,t = w$ available, at the cost of larger keys and signatures. Whether those optimisations can coexist with an adaptor layer was not assumed but *tested* (section 3.5). The modulus is reused too: the reference NTT fixes $q = 8\,380\,417 \approx 2^{23}$, FIPS 204's own modulus [18], rather than the $q \approx 2^{24}$ of [9] — correctness is

unaffected since $q > 2\gamma$, and only the Module-SIS/LWE margin differs. No module depends on a mode-specific constant, so identical code runs at every parameter set.

## 2.3 Serialization and the on-chain interface

A deployable scheme exchanges *bytes*, not in-memory structs, so a bit-packed wire encoding, a typed codec and a verify-from-bytes entry point were implemented. Two decisions matter. Validation is *asymmetric*: keys and witnesses are checked on decode, a signature is not, so a tampered response is caught by Verify — the entry point the tamper test attacks. And, following FIPS 204 [18], a signature carries not the challenge polynomial $c$ but the digest $\tilde{c}$ from which every consumer re-derives it, under the standard's $\lambda/4$ rule (appendix A.5).

## 2.4 Testing methodology

Four test types cover progressively weaker assumptions about the environment, stated in full in table A.2. Functional tests assume an honest caller and assert the whole adaptor cycle, including the *tripwire* clause — a pre-signature must *fail* basic verification — which is what distinguishes an adaptor signature from a signature. Serialization and tamper tests assume a hostile caller; the known-answer test assumes a different machine, and its digest is what the second implementation is checked against (section 2.5).

## 2.5 An independent second implementation: the Rust port

The complete scheme was implemented a second time in Rust on the same reuse principle, layered on a fixed version of the `fips204` crate [21], with the LAS layer added as new modules. The wire format is byte-identical to the C encoder's by construction (fig. 2.3).

The port serves three purposes. *Cross-validation*: short of a formal proof, the strongest practical argument that a cryptographic implementation is correct is a second one written against the specification and agreeing on pinned vectors. *An*

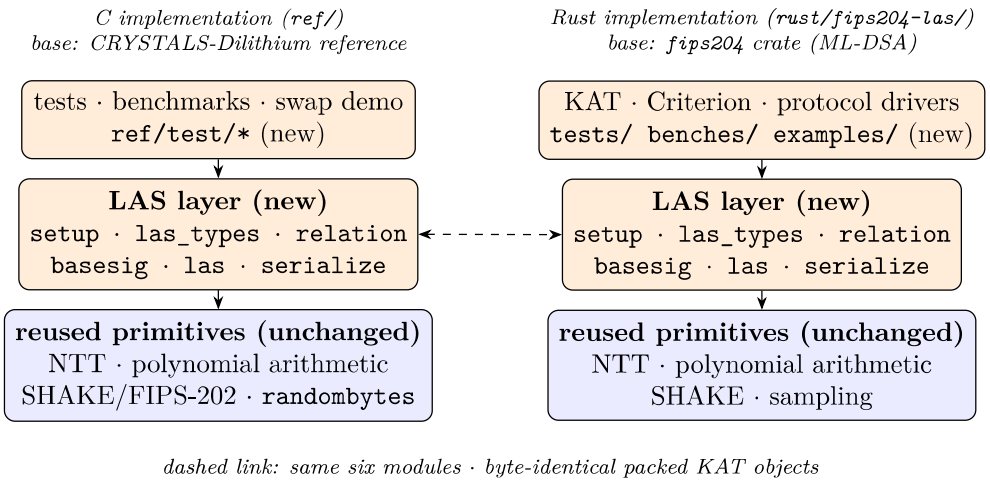


Figure 2.3: Implementation structure. The two implementations follow the same layering: a new, self-contained LAS layer (orange) — the same six modules under the same names — built on an *unmodified* base of reused lattice primitives (blue), with the tests and benchmarks on top. The dashed link is checked rather than asserted: both encoders are anchored to the same packed sizes at compile time, and the packed known-answer objects are byte-identical, which is what the shared pinned digest (`b4a10ffb...03be`) records. This is the high-level map for reading the code; the per-file classification is table A.7.

*independent benchmark methodology*: Criterion.rs [11] shares no timing code with the C harness, so agreement defends the C numbers against harness artefacts. *Deployment realism*: a memory-safe language is the plausible production vehicle.

To make the timings comparable the two protocol drivers mirror each other exactly — same repetition scheme, fixed seed, fixed message, success-path gating and rejection statistics. One caveat qualifies the port's independence: where [9] leaves an engineering choice open, Rust adopts the C implementation's, concretely an early-abort rejection check. Sharing that is what makes the two languages time the same algorithm, but it is established by inspecting the two norm checks rather than by the automated evidence, which constrains outputs and rejection rates, not the work profile of a rejected attempt (appendix A.3).

## 2.6 Benchmarking methodology and fair comparison

The evaluation separates *computation* cost (wall-clock time per operation) from *communication* cost (bytes transmitted or stored).

**Measurement boundaries.** Undocumented boundaries are a recognised benchmarking pitfall for lattice signatures [20], so every timing belongs to one of the three declared boundaries of table A.4: a *core tier* isolating the adaptor algebra from encoding, a *packed tier* giving what an on-chain consumer pays, and, for the classical baseline alone, the *hybrid* boundary its library exposes. Comparisons are made only within one boundary, and every base-versus-LAS comparison shows *both*.

**Implementation of record, repeatability, and machine.** Unless a caption says otherwise, timings are the C implementation's, with the Rust results reported separately and never averaged with them (table 3.4). Every timing is a mean $\pm$ sample standard deviation over the repetition scheme its caption states. All non-random inputs are fixed, so variability comes only from rejection sampling and machine noise, never from input selection [20]. Toolchain, hardware and per-table commands: appendix A.1.

**Per-operation reporting, not cumulative.** Every operation is timed independently: they are split across parties, so a combined figure would average over costs never paid together and hide the per-operation overhead.

**The primary comparison: LAS against its own simplified base.** LAS is exactly its base signature plus four operations on the same code, parameters and primitives, so every comparison holds all three fixed and varies only the adaptor layer (justified in section 4.1). Each adaptor operation is paired with the base operation it mirrors — Adapt against Verify, since it runs a PreVerify before adding the witness — and Extract, having no base analogue, is reported separately.

**Rejection sampling: the expected attempt count.** Sign-class operations restart until the response passes its norm bound, so no timing claim is meaningful

without the attempt statistics. Each coefficient is accepted with probability $\big(2(\gamma-\kappa)+1\big)/(2\gamma+1)$ *independently of the secret* — why the response is simulatable (appendix A.3) — and an attempt succeeds only if all $(n+\ell)\,d$ pass:

$$p_{\text{acc}} = \left(\frac{2(\gamma-\kappa)+1}{2\gamma+1}\right)^{(n+\ell)d} \approx \left(1-\frac{\kappa}{\gamma}\right)^{(n+\ell)d} \approx e^{-\kappa(n+\ell)d/\gamma} = e^{-1} \approx 36.8\%, \tag{2.2}$$

the last step using the design choice $\gamma = \kappa\, d\,(n+\ell)$ from [9], made exactly so the expected number of restarts is "about $e < 3$". Expected attempts are $1/p_{\text{acc}}$, and PreSign's bound being tighter by one, it is *expected* to restart slightly more often before any adaptor arithmetic is counted.

**Rejection sampling: the run-validity gate.** Because the attempt count is geometric, per-call timings are heavy-tailed and averages over too few calls can drift several percent on rejection luck alone [20]. The drivers therefore treat the attempt statistics as a *validity gate*: measured mean attempts must match the closed-form expectation within five standard errors, and a failing run aborts instead of producing evidence. This is a consistency check on the rejection rate, not a proof of sampler correctness (appendices A.2 and A.3). A second gate precedes every timing: the correctness contract is asserted on the objects about to be timed, so no failure path is ever measured.

**Parameter sensitivity and component sizes.** To test whether the adaptor overhead scales with the lattice dimensions, the same comparison is repeated at the four parameter sets of table 2.1 (symbols in table A.3). Each key and signature is decomposed into components, so the *cause* of a size difference can be identified, not merely its total stated.

**The classical baseline: functionality-matched, not security-matched.** The second baseline is the `secp256k1-zkp` library's `ecdsa_adaptor` module [4], implementing Fournier's construction [10], unmodified at a pinned commit. It is a functionality-matched "price of post-quantum" baseline, not a same-security one; plain ECDSA is excluded, the fair counterpart of an adaptor signature being another adaptor signature.

Table 2.1: Parameter sets used in the evaluation. The *LAS-2020/845 reference* set is the parameter set proposed in [9], a historical paper-parameter reference point corresponding to no ML-DSA set; the *Simplified Dilithium-II/III/V* sets reuse the dimensions of Dilithium modes 2/3/5, aligning the reusable ML-DSA primitives and the challenge-digest strength $|\tilde{c}|$ with ML-DSA-44/65/87 while retaining LAS's own ternary secret distribution, rejection bounds, exact hint-free relation and unoptimised serialization. They are engineering benchmark settings, *not* formal NIST/ML-DSA security-equivalence claims: a matched dimension is not a claim that LAS *is* the corresponding ML-DSA parameter set or inherits its security category. All lattice sets share the ring degree $d = 256$ and modulus $q = 8\,380\,417 \approx 2^{23}$ inherited from the reused Dilithium NTT; *Simplified Dilithium-III* is the project's target set. The classical secp256k1 ECDSA adaptor is a functionality-matched baseline at ≈128-bit classical security. Table A.3 defines each symbol.

| Setting | $n$ | $\ell$ | $M$ | $\kappa$ | $\gamma$ | $d$ | $q$ |
|---|---|---|---|---|---|---|---|
| LAS-2020/845 reference | 4 | 4 | 8 | 60 | 122 880 | 256 | 8 380 417 |
| Simplified Dilithium-II | 4 | 4 | 8 | 39 | 79 872 | 256 | 8 380 417 |
| Simplified Dilithium-III (target) | 6 | 5 | 11 | 49 | 137 984 | 256 | 8 380 417 |
| Simplified Dilithium-V | 8 | 7 | 15 | 60 | 230 400 | 256 | 8 380 417 |
| Classical (secp256k1 ECDSA adaptor) | ≈128-bit classical (see table 3.6) | | | | | | |

## 2.7 Application methodology: the UTXO swap study

The narrated demonstration of section 3.6 shows that the two-party atomic-swap protocol of Esgin, Ersoy and Erkin [9] — Figure 1 of that paper, set out in appendix A.6 and called *the swap protocol* here — *runs*. A second study measures what it *costs*, and its conclusions depend on what is held fixed.

**The venue, and why it is modelled.** The construction [9] states its applications over "a UTXO-based blockchain like Bitcoin *where the signature algorithm is replaced with a lattice-based signature scheme*". That sentence is implemented literally: a UTXO ledger was built in which *the signature algorithm is a parameter*, so one ledger serves every configuration and any measured difference is the cryptography, not the ledger. It is a model, not a node; the boundary is set out in appendix A.4.

**What a settled transaction contains, and what the adaptor layer changes.** Because the venue is a transaction rather than a contract, which *components* the construction touches has a precise answer, and fig. 2.4 gives it field by field. Of the objects the swap protocol defines, exactly one reaches a ledger: the adapted signature $\sigma$, in the slot an ordinary signature already occupies — since SegWit [15], a segregated *witness* field. Everything else is an off-chain message, and the witness $y$ is never transmitted at all, $u_2$ deriving it from the published $\sigma$. The adaptor layer thus adds *no on-chain field* — which is what *scriptless* means — so every on-chain byte by which the post-quantum swap exceeds the classical one is attributable to the signature scheme alone (section 3.8).

One notational point matters once the venue is a real ledger. The swap protocol writes the signed object as $tx_i$, but a transaction is not a message: a signature commits to its *signature hash*. Three objects are therefore kept distinct — the unsigned template exchanged off-chain, its signature hash (the message passed to PreSign), and the settled transaction — since conflating them would misreport communication cost.

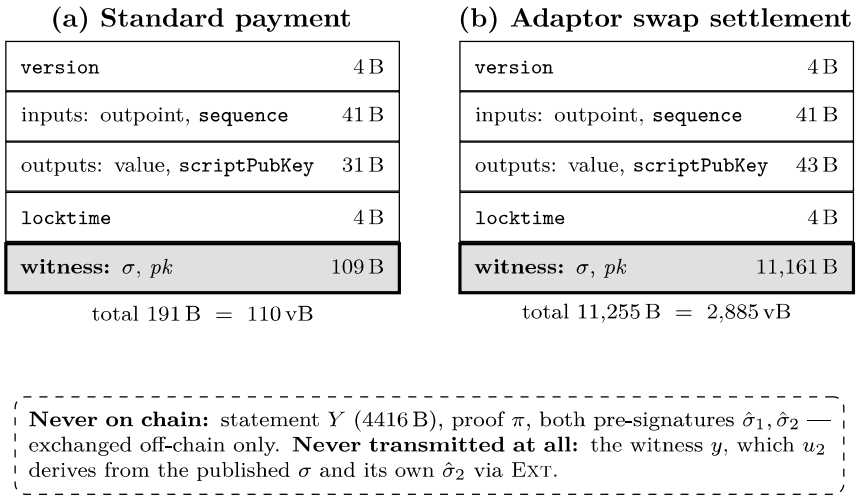


Figure 2.4: The transaction before and after the construction is applied. **No field is added, removed or reordered**: the adaptor layer is invisible on chain, which is what *scriptless* means. Only the two shaded quantities differ, and they differ because the signature scheme changed — the witness grows from 109 B to 11,161 B, and the output script from 31 B to 43 B because it commits to a 32-byte hash of a lattice key rather than a 20-byte hash of an elliptic-curve one. Sizes are the measured objects laid out in the witness serialisation of [13], with the output forms of [15, 23]; table 3.10 breaks them down field by field.

**Three configurations, and what is controlled.** The three configurations of table 2.2 each run the swap protocol end-to-end across two ledgers. The *role-A* proof is the proof of knowledge $\pi$ the protocol places on the wire immediately after $Y$, claiming a witness to $Y$ exists and is short. Only the $2 \to 3$ step is controlled: both LAS configurations share one expansion of the public matrix, derive all key material from one pinned seed, and prove the *identical* relation $\exists\, r' : Ar' = t' \wedge \|r'\|_\infty \leq 1$ — the hard relation of the statement/witness pair, *not* the signer's key pair, and including the norm bound, since proving only $Ar' = t'$ would fail to close the knowledge gap. This is verified rather than assumed: a configuration whose two halves disagree on a *relation binding* each computes independently refuses to run (appendix A.3). The $1 \to 2$ step is not controlled — it changes the signature scheme, the relation and whether a role-A proof exists — so it is a whole-stack comparison, the signature-only question being answered by section 3.4.1.

Table 2.2: The three configurations. Only the $2 \to 3$ step is controlled: those two share one public matrix, prove the identical relation, and receive byte-identical inputs from one pinned seed, so their difference is the proof system alone. Configuration 1 carries *no* role-A proof, and this is a finding rather than an omission: LAS needs $\pi$ because of its knowledge gap — extraction returns a witness of the extended relation, which need not be ternary, so without $\pi$ a malicious $u_1$ could claim and leave $u_2$ holding an unadaptable pre-signature — whereas an ECDSA adaptor has no such gap, since extraction recovers the discrete logarithm exactly. Deployed classical swaps accordingly carry no $\pi$, and adding one would have benchmarked a construction nobody runs. The DLEQ proof carried *inside* a `secp256k1-zkp` pre-signature is a different object and is not counted as $\pi$: its author is the pre-signer, and its claim is pre-signature well-formedness, not knowledge of the role-A statement witness $r'$. Its cost is reported in the bytes by which a pre-signature exceeds a signature.

| # | Signature | Role-A proof $\pi$ | Setup | Fully PQ |
|---|---|---|---|---|
| 1 | ECDSA adaptor signature (libsecp256k1-zkp) | none required | — | no |
| 2 | LAS (simplified Dilithium-III) | Groth16 over BN254 (pairing-based zk-SNARK) | trusted | no |
| 3 | LAS (simplified Dilithium-III) | LaZer (LNP22 linear relation with norms) | transparent | yes |

**The Groth16 circuit.** No pairing-based proof of this lattice statement was available to reuse, so the circuit is new, and cheaper than the statement suggests: $r' \mapsto Ar'$ is *linear with public coefficients*, needing no witness–witness multiplication, the expensive ingredient in a rank-one constraint system (appendix A.3).

**Measurement.** Because gas does not exist on this venue, the axes are execution time and communication cost, the latter counting *off-chain* protocol messages, not only what reaches a ledger. Sizes are observed from the transmitted buffers rather than computed, and these timings sit at the **packed tier**, so they are not comparable with core-tier figures elsewhere. One-time costs are excluded and reported separately (appendix A.3).

# Chapter 3

# Results and discussion

This chapter reports what was measured and why the results should be believed: correctness, computation cost, communication cost, and the application results.

## 3.1 Correctness and robustness

No performance claim means anything until the implementation is shown *correct*, and an adaptor signature is not correct merely because it signs and verifies — it must satisfy a specific contract. Table 3.1 lists every clause; all pass. Clauses 1–4 are the adaptor contract itself: a pre-signature is verifiable yet unspendable (the tripwire), becomes an ordinary signature once the witness is added, and completing it reveals that witness — the mechanism that makes an atomic swap atomic. Clauses 5–6 are robustness, clause 7 reproducibility.

## 3.2 Computation cost

The primary computation result is the cost of the *adaptor layer*. The two paths share algorithm, parameters and primitives, so any difference is purely the price of adaptor functionality (section 2.6). Table 3.2 reports it at *both* boundaries and fig. 3.1 visualises the core tier: every adaptor operation stays within single digits of its counterpart, Extract is the cheapest of all, and every operation completes in well under two milliseconds, signing and pre-signing dominating because of rejection sampling.

At the core boundary the adaptor layer is almost free, and the reasons are structural. PreSign (+6.6% over Sign) runs the same reject loop, adding only $Y$

Table 3.1: The adaptor-signature correctness contract at the target setting, with the test that checks each clause in the evidence run of record (`20260804_101750`); all pass. Clauses 1–4 and 5a are asserted on every one of the 1000 randomised iterations of the functional suite `test_las`; 5b and 6 by `test_serde`, which also performs 256 randomised serialization round trips; 7 by the known-answer test `test_kat`, whose pinned cross-language digest binds the packed public key, secret key, signature, pre-signature and adapted signature over four fixed vectors. PreVerify and Extract are asserted in all three tests, but produce no object the digest absorbs.

| # | Property | Result |
|---|---|---|
| 1 | PreSign produces a pre-signature that PreVerify accepts | pass |
| 2 | the pre-signature *fails* basic Verify (statement-binding tripwire) | pass |
| 3 | Adapt yields a signature that basic Verify accepts | pass |
| 4 | Extract recovers the witness *exactly* ($y' = y$) | pass |
| 5a | a tampered message fails Verify | pass |
| 5b | each of the 6736 low-bit flips of a packed-signature byte is rejected | pass |
| 6 | malformed bytes rejected on decode ($pk \geq Q$, ternary code-3), out-of-range objects on encode ($z$, non-ternary secret) | pass |
| 7 | the deterministic API is byte-identical on re-run | pass |

to the commitment and a tighter bound; PreVerify (+4.0%) recomputes the same commitment shape with the extra $+Y$; Adapt (+6.9%) runs a PreVerify then adds the witness vector; Extract merely subtracts $z - \hat{z}$ and checks $A\,y = Y$. None changes the asymptotic work. This is the headline of the standalone-signature evaluation: *at the core tier* adaptor functionality costs single-digit percentages at every parameter set, and the two operations unique to an adaptor signature cost no more than a verification each (fig. 3.2).

At the packed boundary the premium grows to +20.7%, +38.3% and +90.0% — an order of magnitude higher, and the gap is the statement's decode rather than adaptor arithmetic. The object dominating *communication* cost thus reappears as the dominant *computation* cost once bytes are included, so a deployment verifying from bytes should budget for the packed row — and reported lattice timings are notoriously sensitive to this boundary [20].

Signing and pre-signing are dearer than verification because of rejection sampling, intrinsic to the Fiat–Shamir-with-aborts family rather than a defect here. Equation (2.2) predicts 2.719 attempts per Sign and 2.775 per PreSign, the latter restarting about 2% more often because its bound is tighter by one; counted

Table 3.2: The primary computation result: per-operation adaptor overhead at the target setting *Simplified Dilithium-III* ($n$=6, $\ell$=5, $\kappa$=49, $\gamma$=137 984, $d$=256), at *both* measurement boundaries of section 2.6. Each LAS adaptor operation is paired with the basic operation it mirrors; "Overhead" is the extra cost as a percentage of that basic operation, within the same tier. All timings are measured on the C implementation, the implementation of record (section 2.6); the Rust port's independent measurements are in table 3.4. Timings are $\mu$s per operation (mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500/1000 iterations; machine of record in section 2.6). KeyGen, Sign, and Verify are reused unchanged by LAS; Extract has no basic analogue. The packed-tier overheads are larger because each adaptor operation additionally decodes the public-key-sized statement $Y$ (4416 B) — serialization, not adaptor arithmetic. Absolute core-tier timings at every parameter set are in table A.5 (appendix).

| Operation (basic / adaptor) | Basic ($\mu$s) | LAS adaptor ($\mu$s) | Overhead |
|---|---|---|---|
| *Core tier (structures in/out — isolates the adaptor arithmetic)* | | | |
| KeyGen | $50.9 \pm 0.9$ | $50.9 \pm 0.9$ (shared) | — |
| Sign / PreSign | $489 \pm 17$ | $522 \pm 32$ | +6.6% |
| Verify / PreVerify | $111 \pm 3$ | $115 \pm 2$ | +4.0% |
| Verify / Adapt | $111 \pm 3$ | $118 \pm 2$ | +6.9% |
| Extract | — | $43.7 \pm 0.7$ | (LAS only) |
| *Packed tier (wire bytes in/out — incl. decode + encode)* | | | |
| KeyGen | $167 \pm 0.25$ | $167 \pm 0.25$ (shared) | — |
| Sign / PreSign | $732 \pm 25$ | $883 \pm 19$ | +20.7% |
| Verify / PreVerify | $362 \pm 6$ | $500 \pm 3$ | +38.3% |
| Verify / Adapt | $362 \pm 6$ | $687 \pm 18$ | +90.0% |
| Extract | — | $506 \pm 1$ | (LAS only) |

directly rather than estimated from a timing ratio, the run of record observed 2.615 and 2.724 (table 3.3). At 2500 timed calls the gate band ($\pm\sim$8%) is wider than that 2% separation, so the core-tier PreSign overhead (+6.6%) should be read as of the same order as the prediction rather than as a precise constant; only the $\approx 10^5$-call Criterion harness resolves it.

## 3.3 Cross-implementation check: C implementation versus Rust port

The scheme exists twice (section 2.5), and the two implementations agree at every level at which they can be compared. At the byte level agreement is exact: the same packed sizes (public key 4416 B, signature 6736 B, checked

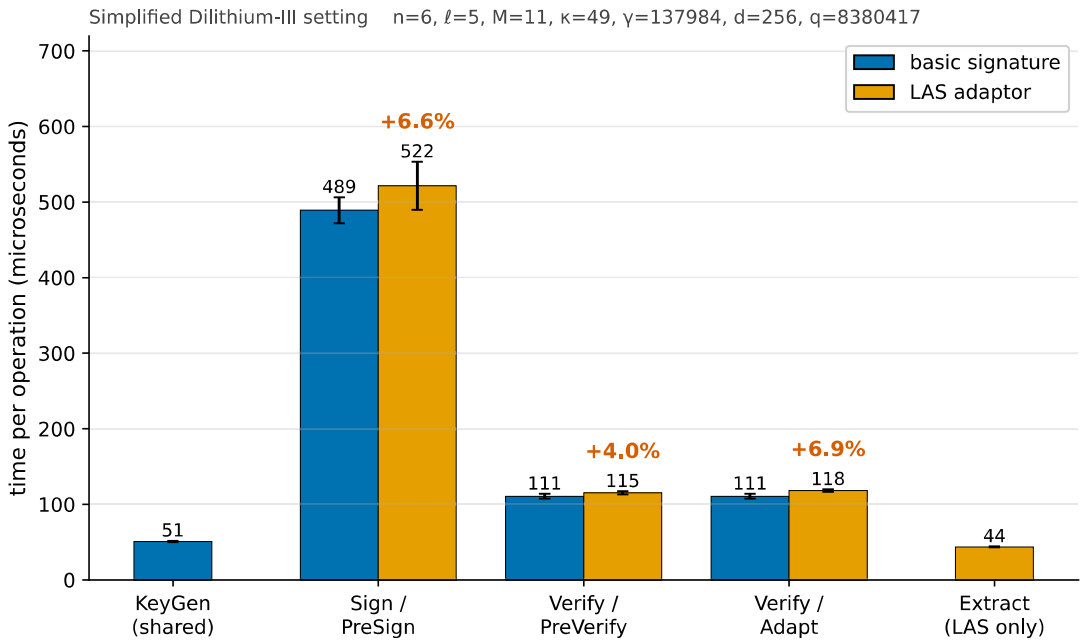


Figure 3.1: The primary computation result, shown visually: each LAS adaptor operation (orange) paired beside the basic operation it mirrors (blue), at the target setting *Simplified Dilithium-III* (its parameters are annotated above the plot), measured on the C implementation (the implementation of record, section 2.6). The percentage above each orange bar is the adaptor overhead over its blue partner; exact means and standard deviations, and the packed tier, are in table 3.2. KeyGen, Sign, and Verify are reused unchanged by LAS, so KeyGen is shown once (shared) and Sign/Verify are the blue partners of PreSign/PreVerify/Adapt; Extract has no basic analogue and is shown LAS-only. The absolute $\mu$s scale also shows the shape: every operation is sub-two-milliseconds and the rejection-sampling operations (Sign, PreSign) dominate, while the verify-class operations and Extract are far cheaper. Error bars are the sample standard deviation over 5 repetitions of 500 (sign-class) / 1000 (verify-class) iterations on the machine of record (section 2.6); the same comparison across all four parameter sets is the scaling check in fig. 3.2.

programmatically on every Rust run) and the same pinned known-answer digest (`b4a10ffb...03be`). Since that digest binds the packed key pair, signature, pre-signature and adapted signature, a divergence escapes it only by leaving every one of those bytes unchanged.

Timing agreement is looser, the builds going through different compilers, but close: every operation agrees within about 18% (largest gap Adapt). More importantly the central conclusion reproduces in both languages. At the core tier the Rust port measures $+2.9\%$, $+-0.7\%$ and $1.8\%$ — all single-digit, like the C

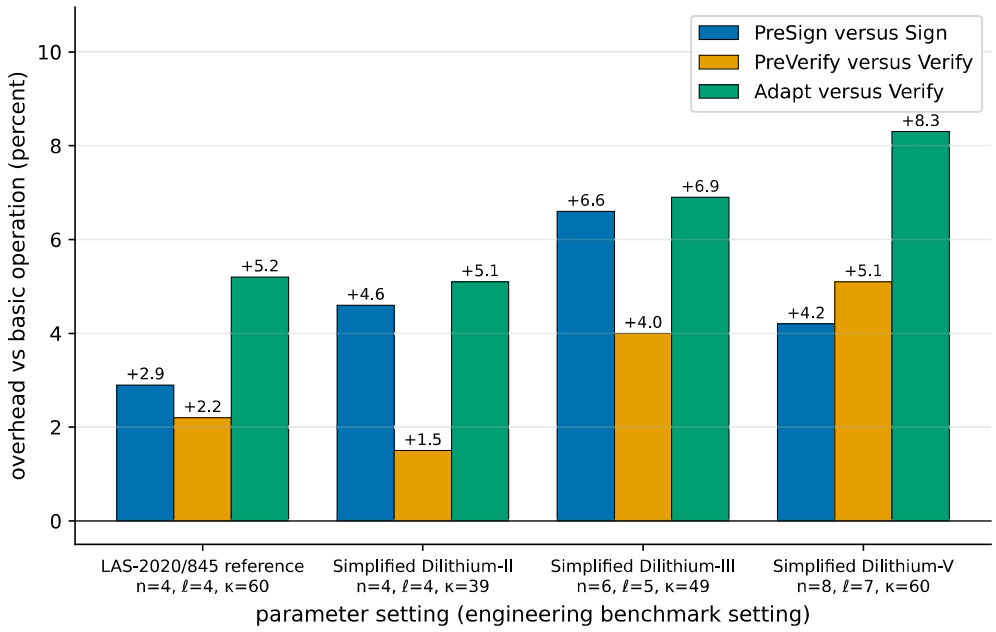


Figure 3.2: Parameter sensitivity of the adaptor overhead: each LAS adaptor operation as a percentage over the basic operation it mirrors, at the four parameter sets of table 2.1. PreSign is paired with Sign; PreVerify and Adapt with Verify; Extract has no basic analogue and is omitted. The core-tier premium stays in the single digits everywhere, so adding adaptor functionality does not become more expensive as the scheme is scaled up. This is a secondary robustness check on the headline of fig. 3.1, not a result about scaling: the primary comparison is the per-operation overhead at the target setting, table 3.2. The absolute timings behind every bar are table A.5 (appendix).

column beside them; the slightly *negative* Adapt figure is the measurement floor rather than a real speed-up, the genuine overhead at this scale being smaller than the gap between the two verify paths. The packed tier reproduces too (+5.3%, +16.8%, +32.1%), all positive and in the C figures' order. Both ports pass the same rejection gate (fig. 3.4). The headline results are therefore properties of the algorithm, not of one codebase, compiler or harness.

## 3.4 Communication cost and component breakdown

The computation results show the adaptor layer is cheap at the core boundary; the size results show where the real cost lies. Table 3.5 decomposes every LAS

Table 3.3: Rejection-sampling statistics at the target setting *Simplified Dilithium-III*: the closed-form geometric model of eq. (2.2) against every measured sample. "Mean" is attempts per call and "Accept." is the per-attempt acceptance rate — the two quantities every source records, and the ones the model predicts. The per-call *dispersion* (the geometric standard deviation $\sigma = E\sqrt{1 - 1/E}$, the medians and 95th percentiles, and the sampled maxima) is annotated on the companion acceptance curve of fig. 3.3 rather than repeated here. The C driver is the implementation of record; the Rust protocol driver and the Criterion harness are the independent samples of section 2.5. Every measured row passes the five-standard-error validity gate of section 2.6.

| Operation | Source (calls) | Mean | Accept. |
|---|---|---|---|
| Sign | geometric model, eq. (2.2) | 2.719 | 36.8% |
| | C driver, distribution sample (2000) | 2.615 | 38.2% |
| | C driver, timed-run gate (2500) | 2.756 | 36.3% |
| | Rust driver, timed-run gate (2500) | 2.704 | 37.0% |
| | Rust Criterion, gate (188791) | 2.735 | 36.6% |
| PreSign | geometric model, eq. (2.2) | 2.775 | 36.0% |
| | C driver, distribution sample (2000) | 2.724 | 36.7% |
| | C driver, timed-run gate (2500) | 2.852 | 35.1% |
| | Rust driver, timed-run gate (2500) | 2.766 | 36.1% |
| | Rust Criterion, gate (188791) | 2.784 | 35.9% |

object into bytes at the target setting, explaining *which* component drives the size. Sizes are fixed by the encoding, so they are exact and carry no dispersion.

Two points follow. First, **the response $z$ is essentially the entire signature** — 99.3% of it at every parameter set — since the challenge travels only as its short digest, so any optimisation of LAS signatures is an optimisation of $z$. Second, **an adaptor swap additionally transmits a public-key-sized statement $Y$ and a signature-sized pre-signature**: the only objects an adaptor scheme sends that a basic signature does not.

**The sharp conclusion: the cost of LAS is not adaptor computation but communication — specifically $z$ and $Y$.** The adaptor operations add at most +8.3% of compute, while the objects on the wire are kilobytes. That matters more in a blockchain setting than "the signature is 6736 bytes": bytes reaching a ledger are stored and replicated by every node, while the statement and pre-signature, though off-chain, still cost both parties bandwidth. The response is stored at full width, neither range-reduced nor entropy-coded — the clearest opportunity for reducing that footprint (section 5.3).

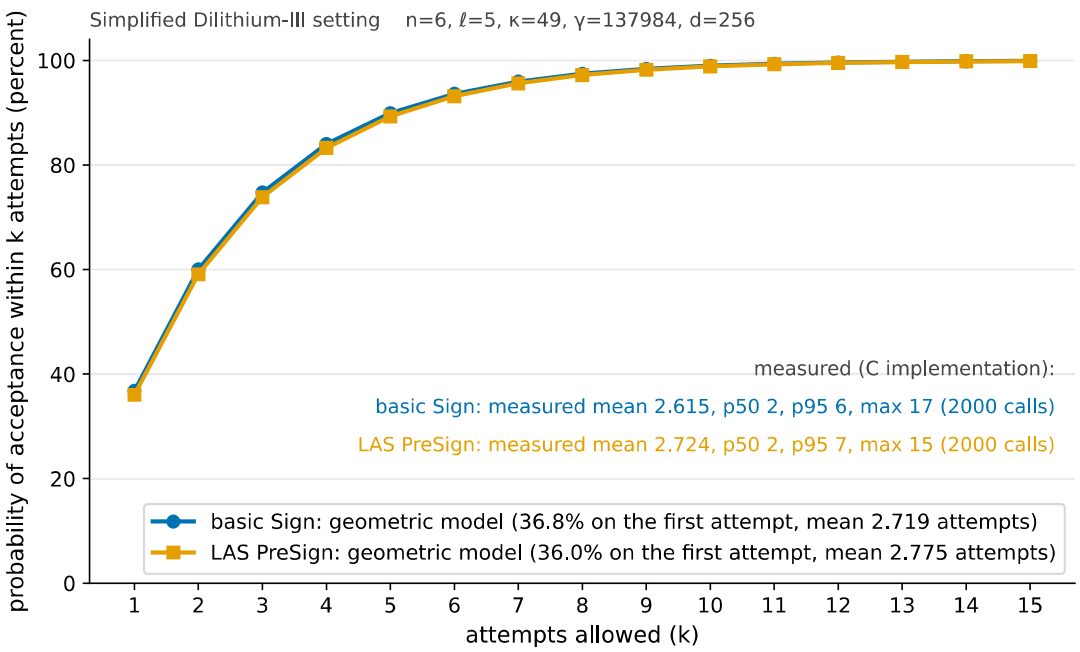


Figure 3.3: The probability that a sign-class call has been accepted *within k* rejection-sampling attempts, at the target setting *Simplified Dilithium-III*. The curves are the closed-form geometric model of eq. (2.2) for the basic Sign bound $\gamma - \kappa$ (blue) and the tighter LAS PreSign bound $\gamma - \kappa - 1$ (orange): acceptance is 36.8% and 36.0% respectively on the first attempt, passes 90% by five attempts, and then flattens — the two curves are nearly coincident, which is exactly why the PreSign-versus-Sign timing gap is so small. The annotated statistics are *measured* on the C implementation (the 2000-call distribution sample tabulated exactly in table 3.3). The flattening is the practical point: allowing many more attempts buys almost nothing, because the overwhelming majority of calls are accepted within the first few.

### 3.4.1 The price of post-quantum: classical adaptor baseline

The second baseline is a classical secp256k1 ECDSA adaptor signature [4]: the natural functional counterpart of LAS, driving the same scriptless swaps but resting on discrete-log rather than lattice hardness. It measures the practical cost of moving from a classical adaptor signature to a post-quantum one. LAS is instantiated at *Simplified Dilithium-II* and only there: secp256k1 offers roughly 128-bit classical security, so those dimensions are the most like-for-like engineering match, while -III or -V would pit the classical scheme against a deliberately stronger configuration. The boundaries differ between the two libraries, so table 3.6 matches them per operation and the comparison is read conservatively — the conclusions

Table 3.4: Per-operation timing of the two implementations at the target setting, *Simplified Dilithium-III* ($\mu$s per operation). The C and Rust columns come from the two protocol drivers, which deliberately mirror each other (same repetition scheme of 5 repetitions × 500/1000 iterations, fixed public-parameter seed, fixed message, same success-path gating), so they are directly comparable. The final column is the independent Criterion.rs statistical harness (300 samples per operation, bootstrap 95% confidence interval of the mean); its hot-loop protocol yields lower absolute times, and it is included to show that the *relative* conclusions do not depend on the hand-rolled driver.

| Operation | C implementation ($\mu$s) | Rust port ($\mu$s) | Rust / C | Rust, Criterion ($\mu$s, 95% CI) |
|---|---|---|---|---|
| KeyGen | $50.9 \pm 0.9$ | $51.5 \pm 0.5$ | 1.01 | 41.3 [41.2, 41.4] |
| Sign | $489 \pm 17$ | $438 \pm 14$ | 0.90 | 377 [375, 378] |
| Verify | $111 \pm 3$ | $95.4 \pm 0.5$ | 0.86 | 75.6 [75.3, 75.9] |
| PreSign | $522 \pm 32$ | $451 \pm 7$ | 0.86 | 386 [384, 388] |
| PreVerify | $115 \pm 2$ | $94.8 \pm 0.9$ | 0.82 | 76.8 [76.4, 77.1] |
| Adapt | $118 \pm 2$ | $97.1 \pm 1.4$ | 0.82 | 77.9 [77.5, 78.3] |
| Extract | $43.7 \pm 0.7$ | $37.8 \pm 0.2$ | 0.87 | 28.5 [28.4, 28.6] |

rest on order-of-magnitude gaps, not percent-level differences.

Two findings stand out. First, **the post-quantum price is paid almost entirely in size**: LAS's signature and public key are roughly 72× and 89× larger than the classical adaptor's, while every LAS operation stays well under a millisecond. Even the largest per-operation factor (Adapt, 286×) repays decomposition, because it compares two different *quantities of work* rather than two speeds for the same work: Algorithm 2 of [9] obliges Adapt to run PreVerify before returning, so 69% of LAS's 443 $\mu$s is a full lattice verification and the rest is largely codec, whereas `secp256k1_ecdsa_adaptor_decrypt` merely inverts one scalar and multiplies, leaving verification to a separate exported call [4]. Charging the classical side the verification LAS cannot skip puts the two at 119 $\mu$s against 443 $\mu$s — 4×, not 286×: a property of the two *interfaces*, not of lattice arithmetic. Second, **the two schemes pay their adaptor overhead in opposite ways**. The classical adaptor must attach a discrete-logarithm-equality proof [10], making its PreSign and PreVerify several times its own base operations, whereas LAS folds the statement into a hash it already computes. The lattice adaptor layer is the cheaper one to add.

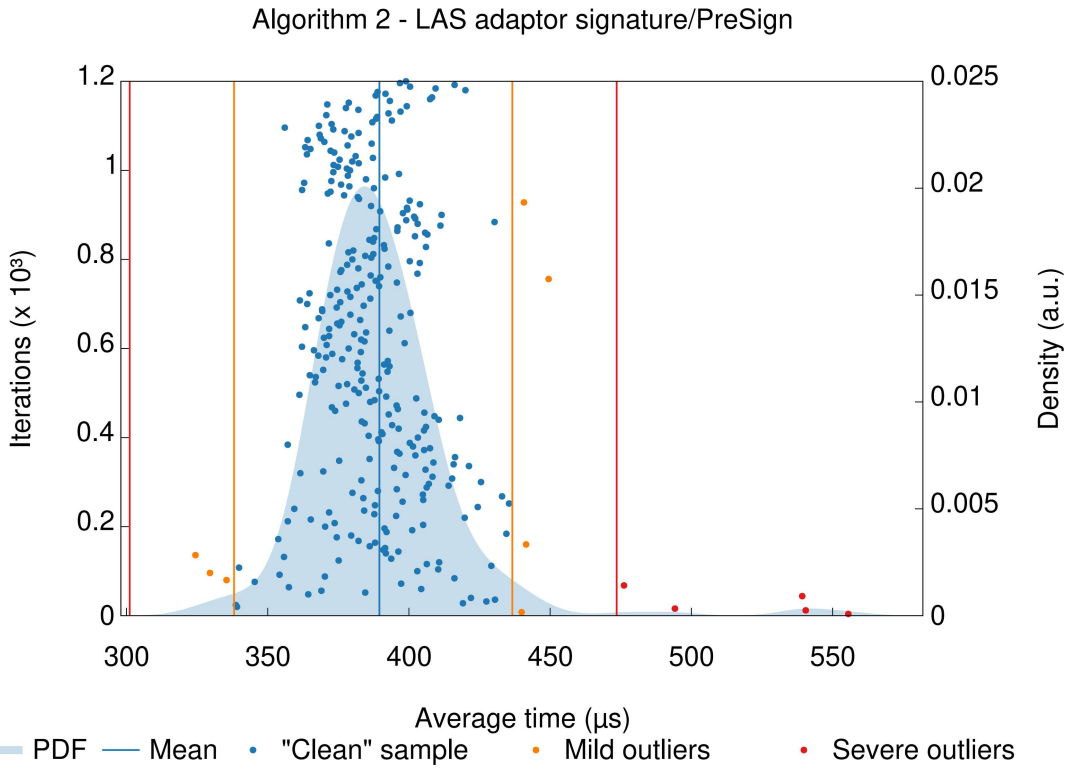


Figure 3.4: The statistical harness's own report for PreSign at the target setting, reproduced from Criterion.rs: 300 timed samples (dots), the kernel-density estimate of the sampling distribution (shaded), the mean (vertical blue line), and Criterion's automatic outlier classification (mild / severe, orange / red). Four presentational changes were made to Criterion's SVG on the way to PDF — the type enlarged, the legend moved from the right-hand column into the row beneath the plot, the margins re-flowed and cropped to the result, and gnuplot's three-part exponent rendered as the single glyph $10^3$; inside the plot frame the coordinate map is the identity, so no plotted value, curve, mean, axis scale or outlier classification is altered. This is the evidence behind the Criterion column of table 3.4: a well-formed, single-mode distribution whose spread comes from rejection sampling and scheduling noise, with only a handful of classified outliers.

## 3.5 Testing the simplification: the adaptor on unmodified ML-DSA

Section 2.2 disabled Dilithium's hint vector, Power2Round and high/low-bit decomposition, reasoning that the adaptor needs the exact identity $A\mathbf{z}-ct = w+Y$. That was an *assertion*, so it was tested: the four adaptor operations were built a third time on NIST ML-DSA *as FIPS 204 specifies it* [18], all three optimisations

Table 3.5: Serialized size of every LAS object at the *Simplified Dilithium-III* (target) setting, in packed wire bytes (not on-chain gas), with its paper notation. "In the basic signature?" marks which objects LAS shares with the base scheme and which it adds. Three facts read off directly: the signature, pre-signature, and adapted signature are byte-identical (adaptation changes the *value* of the response, not its size); the response $z$ dominates every signature; and the only extra *public* object LAS adds over a basic signature is the statement $Y$, the size of a public key. The sizes hold for the C and the Rust implementation alike: the wire encoding is byte-identical in both by construction, asserted programmatically on every Rust run (section 3.3). Sizes at every parameter set: table A.6 (appendix).

| Object | Notation | Bytes | In the basic signature? |
|---|---|---|---|
| Public key | $pk = t$ | 4416 | yes (shared) |
| Secret key | $sk = r$ | 704 | yes (shared) |
| Challenge | $c$ | 48 | yes |
| Response | $z$ / $\hat{z}$ | 6688 | yes |
| Signature | $(c, z)$ | 6736 | yes (basic signature) |
| Statement | $Y = t'$ | 4416 | LAS only (public) |
| Witness | $y = r'$ | 704 | LAS only (private) |
| Pre-signature | $(c, \hat{z})$ | 6736 | LAS only |
| Adapted signature | $(c, z)$ | 6736 | LAS (verifies as basic sig) |

enabled. Three findings follow; the second overturns the assertion.

**The naive port fails outright.** Adding the statement to the challenge hash and changing nothing else — treating the hint machinery as a black box — yields pre-signatures that do not pre-verify at all (0/200), because the transmitted hint reconstructs HighBits($w$) while the challenge committed to HighBits($w + Y$). Some modification is therefore genuinely unavoidable.

**But the verifier is not what must change.** Once the *whole* commitment path moves onto $w + Y$ — the committed high bits, the low-bits rejection test *and* MakeHint — and PreSign tightens its bound by the witness norm, the construction satisfies the full adaptor contract (13/13 items, including four tamper rejections and byte-identical determinism), and, decisively, **the unmodified FIPS 204 verifier accepts the adapted signature** (200/200). So PreSign and PreVerify are necessarily new algorithms, while Verify is not: a post-quantum adaptor swap could settle against a standard ML-DSA verifier. A matched no-statement baseline reproduces FIPS 204's own published repetition rates, which is what makes these rows attributable to the adaptor rather than to a porting error.

Table 3.6: Classical vs. post-quantum adaptor signature: same functionality, different security foundation. Classical secp256k1 (≈128-bit *classical* security) against LAS at the *Simplified Dilithium-II* parameters of table 2.1. Those dimensions are an engineering match to the ≈128-bit (NIST level 2) target, *not* a formal security-equivalence claim; the stronger LAS settings are reserved for the parameter-sensitivity study (section 3.2) and would overstate the post-quantum cost here. Timings $\mu$s/op (mean ± SD, machine of record); sizes in bytes. The two libraries draw their measurement boundaries differently, so the final column is *tier-matched* — each LAS row taken at whichever of its two tiers corresponds to the classical library's boundary for that operation. Why that matching is necessary, the boundaries themselves, the repetition counts, and the object-level differences behind the size rows are set out in appendix A.3.

| | ECDSA adaptor (classical, hybrid native API) | LAS adaptor (post-quantum, core tier) | LAS adaptor (post-quantum, packed tier) | overhead LAS ÷ classical (tier-matched) |
|---|---|---|---|---|
| *Computation ($\mu$s/op)* | | | | |
| KeyGen | $14.4 \pm 0.03$ | $34.1 \pm 0.1$ | $116 \pm 1$ | 2.4× |
| Sign | $19.9 \pm 0.1$ | $314 \pm 14$ | $494 \pm 4$ | 16× |
| Verify | $29.6 \pm 0.3$ | $74.1 \pm 0.7$ | $134 \pm 18$ | 2.5× |
| PreSign | $91.2 \pm 1.2$ | $328 \pm 19$ | $576 \pm 11$ | 6.3× |
| PreVerify | $117 \pm 1$ | $75.2 \pm 0.6$ | $305 \pm 8$ | 2.6× |
| Adapt | $1.6 \pm 0.02$ | $77.8 \pm 0.2$ | $443 \pm 3$ | 286× |
| Extract | $16.0 \pm 0.1$ | $28.4 \pm 0.1$ | $323 \pm 5$ | 20× |
| *Communication (bytes)* | | | | |
| Public key / statement | 33 | 2944 | | 89× |
| Secret key / witness | 32 | 512 | | 16× |
| Signature | 64 | 4640 | | 72× |
| Pre-signature | 162 | 4640 | | 29× |

**The size gain is real but bounded, and it relocates the bottleneck.** Measured against the scheme of record in one process, the adaptor layer stays single-digit on both constructions (+2.8% against +2.2% for PreSign per attempt), and per-signature cost is close to equal for opposite reasons: ML-DSA restarts about twice as often (5.2172 against 2.7136) but each attempt is roughly half the price. What ML-DSA buys is *bytes* — the signature falls to 3309 B from 6736 B — but the statement $Y$ does not shrink at all (byte-identical at 4416 B), because Power2Round compresses a public key while $Y$ enters the verification identity *before* any rounding. The swap payload therefore improves only to 0.69×, and $Y$ becomes larger than the signature it accompanies. **Compressing the**

**signature moves the bottleneck to the statement rather than removing it**, redirecting the size question of section 3.4 and the future work of section 5.3.

This is a functional demonstration, not a security argument: whether committing to HighBits$(w + Y)$ preserves ML-DSA's unforgeability is not analysed, and is exactly the kind of claim section 4.4 declares out of scope.

## 3.6 Application demonstration

The implementation drives an end-to-end post-quantum atomic swap following the swap protocol [9] *verbatim*, including its proof of knowledge $\pi$ (fig. 3.5 and appendix A.6). Two results matter. First, the protocol runs and every step is asserted, including the counterfactuals that a pre-adaptation signature fails basic verification and that $\pi$ is rejected against any other statement. Second, $\pi$ is what makes the second party's final step *guaranteed* rather than merely expected: by the Module-SIS uniqueness argument of [9] the extracted witness equals the proven ternary one, so the adapted signature is certain to clear the verification bound — without $\pi$ a malicious counterparty could claim and leave an unadaptable pre-signature behind. The proof measures 32 046 B at a knowledge error below $2^{-127}$ and is exchanged off-chain only.

As a supporting check on deployability, native on-chain LAS verification was *implemented* and measured on a real Solidity stack: one claim transaction running full verification and releasing one leg's escrow costs ≈56.6 million gas (table 3.7; fig. 3.6), ≈748× the classical claim and over the EIP-7825 cap. An *optimistic* (Naysayer) alternative makes the honest path cheap (≈1.2 million) but leaves a worst-case fraud proof at ≈28.2 million, also over the cap, and a fraud proof that cannot be mined is not one (appendix A.7). These measurements are why the application was taken to a UTXO chain.

That figure measures one *implementation*, not the scheme. A restructured verifier — same parameters, bounds and challenge preimage, differing in how the computation is expressed — was mined by a real client, running full LAS verification and releasing the escrow at 16 413 275 gas: 97.8% of the EIP-7825 cap, which bounds the transaction gas limit and so covers calldata as well as execution. Its equivalence is conditional on a registration invariant, and both are set out in table 3.7. On-chain verification therefore fits in one transaction at *Simplified Dilithium-III* for the 32-byte signed message measured, with a thin

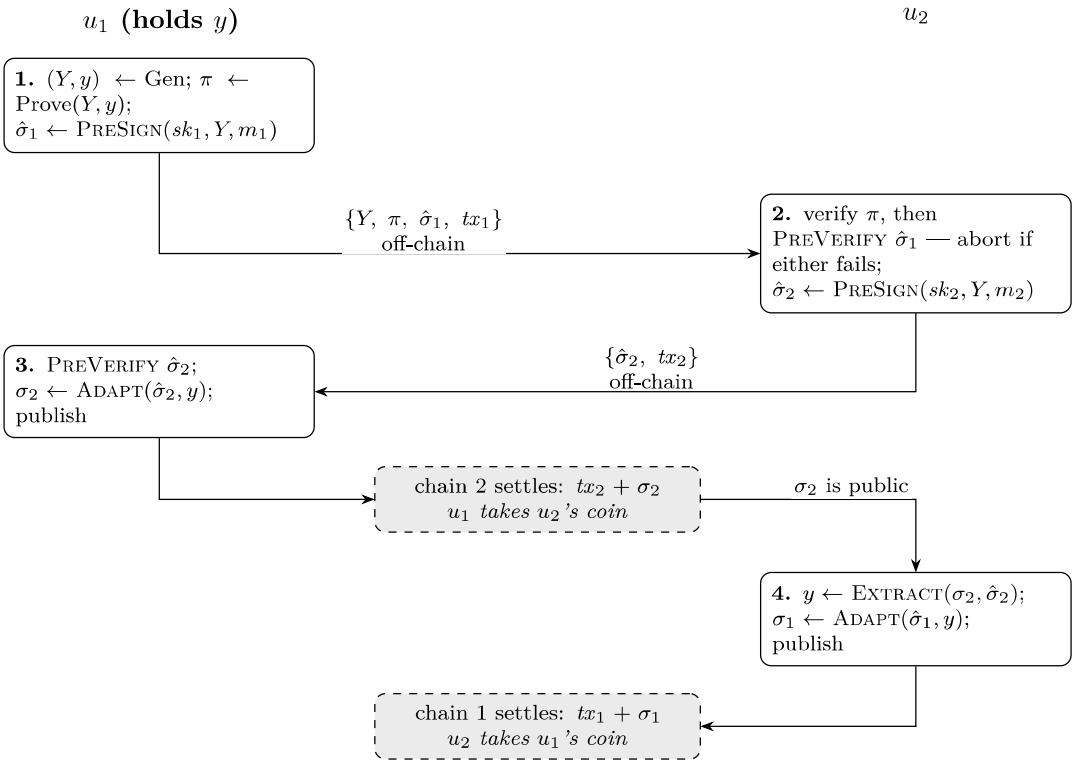


Figure 3.5: The swap protocol as implemented, with every object the two parties exchange. $u_1$ generates the statement/witness pair, so it moves first and claims first. The signed message $m_i$ is the signature hash of the unsigned template $tx_i$, not the transaction itself (section 2.7). Step 2 is the abort gate: $u_2$ commits nothing until both $\pi$ and $u_1$'s pre-signature verify, and at that point neither pre-signature is spendable by anyone. Step 4 needs no message from $u_1$ at all — $u_2$ reads $\sigma_2$ off chain 2 and subtracts its own pre-signature from it, and that leak is what makes the exchange atomic rather than merely simultaneous. Only the two dashed boxes reach a ledger: the statement, the proof and both pre-signatures never do, and the witness $y$ is transmitted by nobody. The two off-chain arrows and the two dashed boxes are exactly the message rows of table 3.9, and the operations named in the boxes are the phases timed in table 3.8.

margin that is a message-length budget rather than slack. The change of venue stands: the UTXO constraint is a transaction *weight* limit one settlement sits well inside (section 3.8).

Table 3.7: On-chain gas for the swap's claim step, measured on a local EVM (`forge -gas-report`); EVM gas is deterministic for fixed bytecode, revision, inputs and state. **These are *execution* figures** — they exclude the 21,000 intrinsic charge and the per-byte calldata a real transaction also pays under EIP-7623, which is why the under-the-cap claim in the text rests on a client receipt rather than on this table. `claimLAS` performs *no* lattice verification and is a strict lower bound; ECDSA verifies in the native `ecrecover` precompile, whereas both LAS rows run in EVM bytecode. The two `claimLASVerified*` rows decide the same predicate as `base_verify` (section 2.1), the second restructuring that computation rather than changing it — but **only under a registration invariant that neither row checks**, and where it fails a *different* predicate is decided; the equivalence testing, the invariant and the cost of breaking it are set out in appendix A.3. The client run reported in the text used anvil at a pinned `osaka` revision with EIP-7825 enforced, over 50 148 B of calldata. **Scope**: *Simplified Dilithium-III*, a 32-byte signed message, that revision; no other parameter set was evaluated, the verifier's dimensions being fixed at build time. The 363 941 gas of headroom is *measured*; that it corresponds to a few hundred further bytes of signed message is *derived* from it and the measured per-permutation hash cost.

| Claim path | On-chain verification | Gas | × classical |
|---|---|---|---|
| Classical ECDSA adaptor (`claimClassical`) | full, `ecrecover` precompile | 75 763 | 1× |
| LAS floor (`claimLAS`) | none (calldata + `keccak256`) | 290 640 | 3.8× |
| LAS full verify (`claimLASVerified`) | full `base_verify`, in Solidity | 56 647 414 | 748× |
| LAS full verify, restructured (`claimLASVerifiedOpt`) | full `base_verify`, in Solidity | 16 438 275 | 217× |

## 3.7 The swap on a UTXO chain: three configurations compared

The demonstration above establishes that the protocol runs; this section measures what it *costs*, over the UTXO ledger and under the design of section 2.7: three configurations (table 2.2), of which only the $2 \to 3$ step is controlled. Timings are the mean ± sample standard deviation over 10 complete swaps at the *packed* tier, so they are not comparable with section 3.2. Both LAS configurations passed the rejection gate (2.8510 attempts per PreSign against the closed-form 2.77483).

**The proof, not the signature, is the cost of a post-quantum swap.** The role-A proof consumes 99.3% of end-to-end time in configuration 2 and 98.6%

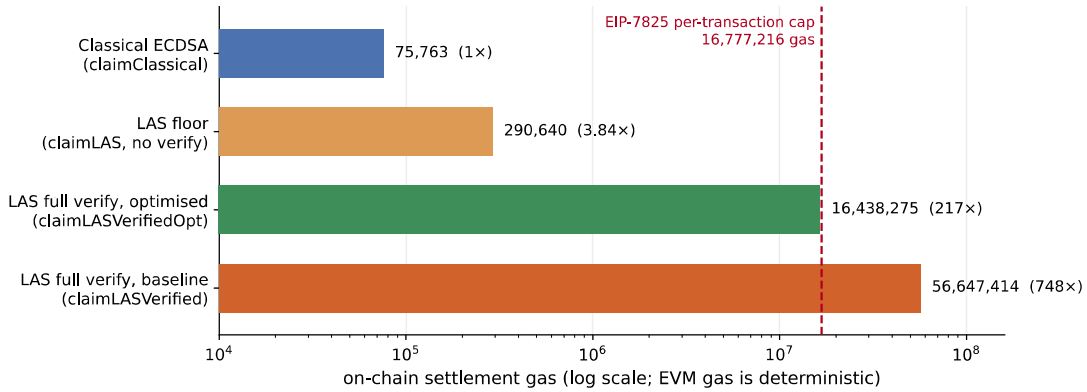


Figure 3.6: The claim paths of table 3.7 on a logarithmic axis, which is what makes the spread legible. The dashed line is the EIP-7825 cap on the transaction gas limit ($16\,777\,216 = 2^{24}$): the baseline verifier's bar crosses it, while the restructured `claimLASVerifiedOpt` — same scheme, same predicate under the registration invariant of table 3.7, different expression — falls below. All four bars come from one `-gas-report` table and so are mutually comparable; the under-the-cap claim itself rests on the separate real-client run described in table 3.7.

in configuration 3. Strip it out and the entire post-quantum signature workload — two key generations, a statement, two pre-signatures, a pre-verification, two adaptations, an extraction and two settlements — costs 4468.6 $\mu$s, about 5.5× the classical swap's 819.7 $\mu$s. A factor of five on a millisecond-scale workload is unremarkable; a proof of hundreds of milliseconds decides whether the protocol feels instantaneous. Even at the packed tier the signature workload is not the bottleneck — the post-quantum penalty is not where intuition puts it.

**The controlled comparison inverts the expected trade-off.** Replacing Groth16 with LaZer — same signature, same $\mathbf{A}$, same inputs — makes the swap *faster* by 53.1% while making it *larger* by 61.9%: roughly twice as quick to produce a proof, yet emitting one about 240× bigger. Groth16 is *succinct* — three group elements whatever the circuit — but producing that proof means a multi-scalar multiplication over the whole constraint system, whereas LaZer's proof grows with the statement yet proves a lattice relation with lattice machinery, avoiding the encoding of a 23-bit modulus into a 254-bit pairing-friendly field. Verification runs the other way (13718.1 $\mu$s against 90720.8), succinctness paying off exactly where it is designed to.

Table 3.8: Per-phase execution time of one complete swap, mean ± sample SD over 10 swaps ($\mu$s; run 20260730_162109, AMD Ryzen 7 7745HX with Radeon Graphics). Dashes mark phases a configuration does not have. End-to-end is context, not the headline.

| Phase | Classical | LAS + Groth16 | LAS + LaZer |
|---|---|---|---|
| KeyGen ×2 + Gen | 58.4 | 540.7 | 503.2 |
| Prove ($\pi$) | — | 645621.6 | 216080.6 |
| PreSign ($u_1$) | 114.9 | 1065.3 | 1065.7 |
| ProofVerify ($\pi$) | — | 13718.1 | 90720.8 |
| PreVerify (abort gate) | 147.6 | 364.9 | 380.2 |
| PreSign ($u_2$) | 107.1 | 1070.1 | 966.7 |
| Adapt ($u_1$) | 138.4 | 392.5 | 380.5 |
| Settle chain 2 | 51.9 | 253.0 | 239.9 |
| Extract ($u_2$) | 25.3 | 185.2 | 183.6 |
| Adapt ($u_2$) | 132.7 | 353.8 | 340.9 |
| Settle chain 1 | 43.4 | 243.0 | 234.9 |
| end-to-end | 819.7 | 663808.3 | 311097.1 |
| of which $\pi$ | — | 99.3% | 98.6% |
| excluding $\pi$ | 819.7 | 4468.6 | 4295.7 |

**Configuration 2's trade is worse than it looks.** Pairing security is broken by Shor, so its proof is precisely the component a post-quantum adversary attacks: a post-quantum signature does not protect a swap whose fairness rests on a classical proof. Groth16 also needs a *trusted, per-circuit* setup (1.225142318s) whose secret must be destroyed for soundness, whereas LaZer's parameters are transparent. Configuration 3 is thus not merely the post-quantum option: it is faster end-to-end, needs no trusted party and removes a quantum vulnerability, paying only in bytes.

**Communication is again the real price, with a usability consequence.** A classical swap moves 941 B; the fully post-quantum swap moves 80059–80108 B, about 85.1× more, and puts 63.1× more on the ledgers — the multiple that matters economically, since a UTXO chain prices by size. A fully post-quantum swap needs 311097.1 $\mu$s and tens of kilobytes of off-chain messaging per exchange, overwhelmingly the proof, against 819.7 $\mu$s classically: comfortable on a laptop, but a wide enough gap that a mobile wallet's assumptions would not port naively. No constrained device was measured, so that point should not be over-read.

Table 3.9: Communication cost of one complete swap, in bytes, observed from the transmitted buffers over 10 swaps. Off-chain messages are counted, not only what reaches a ledger. **A range indicates a message whose size varied between runs, and only configuration 3's proof does so**: LaZer packs the proof's Gaussian response vectors with a variable-length code and then reports the length its encoder actually reached, so the packed size depends on the values sampled for that particular proof. Every other object here has a size fixed by its encoding, and the two ranged totals inherit that single varying term.

| Message | Classical | LAS + Groth16 | LAS + LaZer |
|---|---|---|---|
| *Off-chain (exchanged directly between the parties)* | | | |
| statement $Y$ | 33 | 4416 | 4416 |
| proof $\pi$ | — | 128 | 30711–30760 |
| $\hat{\sigma}_1$ | 162 | 6736 | 6736 |
| $tx_1$ | 114 | 4497 | 4497 |
| $\hat{\sigma}_2$ | 162 | 6736 | 6736 |
| $tx_2$ | 114 | 4497 | 4497 |
| **off-chain total** | **585** | **27010** | **57593–57642** |
| *On-chain (published to the ledgers)* | | | |
| $tx_1 + \sigma_1$ (chain 1) | 178 | 11233 | 11233 |
| $tx_2 + \sigma_2$ (chain 2) | 178 | 11233 | 11233 |
| **on-chain total** | **356** | **22466** | **22466** |
| **total per swap** | **941** | **49476** | **80059–80108** |

## 3.8 Transaction structure and what a UTXO chain would charge

The sizes above are the model ledger's own encoding (appendix A.4). Laid out instead in Bitcoin's wire format, what does a settled swap cost, and does it fit? Table 3.10 answers by projecting the *measured* object sizes onto the serialisation of [15, 13, 23]; the arithmetic is generated, and accepted only if it first reproduces two published reference figures.

**The witness discount absorbs more than half of the post-quantum penalty.** This rests on a transaction structure Bitcoin did not originally have. Before SegWit [15, 13] a signature travelled in the input script, where every byte was billed alike; the segregated witness bills witness bytes at one weight unit against four, and Taproot [23] supplies the output form used here. In raw bytes the post-quantum settlement is 58.9× the classical one; in virtual size — what fees

Table 3.10: One settled swap transaction in Bitcoin's wire format, field by field. The layout is identical in both columns (fig. 2.4); only the sizes differ. Object sizes are measured (run 20260730_162109); the field arithmetic is derived from [15, 13, 23] and self-checked against the published 110 vB P2WPKH reference spend. Virtual size is the quantity a UTXO chain prices, and the gap between it and total size is the witness discount. The model ledger's own encoding (table 3.9) totals 11233 B for the same objects, agreeing with the 11,255 B here to within 0.2 %: the two differ only in where the public key sits — inline in the model's output, in the witness under Bitcoin's layout — and in Bitcoin's marker, flag and compactSize prefixes.

| Field | Classical (B) | LAS (B) |
|---|---|---|
| *Base data — billed at 4 weight units per byte* | | |
| `version` | 4 | 4 |
| input count | 1 | 1 |
| input: outpoint + empty `scriptSig` + `sequence` | 41 | 41 |
| output count | 1 | 1 |
| output: value + `scriptPubKey` | 31 | 43 |
| `locktime` | 4 | 4 |
| **base size** | **82** | **94** |
| *Witness data — billed at 1 weight unit per byte* | | |
| `marker` + `flag` | 2 | 2 |
| witness stack: signature $\sigma$ + public key | 107 | 11,159 |
| **witness size** | **109** | **11,161** |
| **total size** | **191** | **11,255** |
| weight (WU) | 437 | 11,537 |
| **virtual size (vB)** | **110** | **2,885** |

are charged on — only 26.2×, a post-quantum signature being *entirely* witness data. That decision, taken long before anyone considered lattice signatures on Bitcoin, removes 55 % of the size penalty; on the original structure the same settlement would pay the undiscounted rate throughout. It is a concrete argument for UTXO chains as the venue, with no counterpart on the EVM, where calldata carries no comparable discount and native verification cost 56.6 M gas (table 3.7).

**It fits the size limits, but not the verification rules.** A settlement weighs 11,537 WU against the 400,000 WU standardness limit — 2.9 % of it — and the 4,000,000 WU block limit admits an upper bound of 346 settlements per block against 9,153 classical ones, ignoring competing transactions. A UTXO chain does impose a limit, then, but a weight limit rather than a gas limit, and one swap

sits well inside it — the contrast with the EVM, where the first native verifier exceeded the transaction gas cap outright and forced the change of venue.

Two obstacles survive, neither caused by the adaptor layer. No deployed consensus rule verifies a lattice signature: Script offers `OP_CHECKSIG` over ECDSA or BIP-340 Schnorr and nothing else, so configurations 2 and 3 could not settle on Bitcoin as it stands; [9] assumes only the venue, a UTXO chain "where the signature algorithm is replaced with a lattice-based signature scheme", and this study takes no position on which consensus route would deliver it. Separately, the objects collide with limits that exist regardless: the signature is 13.0× over the 520 B consensus stack-item limit and 84× over the 80 B standardness limit [3]. Both obstacles apply equally to an ordinary post-quantum payment: they are properties of lattice signature sizes, not of adaptor signatures.

**Both obstacles were put to a real client.** A stock Bitcoin Core node carries the lattice objects — refused by default relay policy, yet consensus-valid and mined — but verifies nothing: an ordinary Schnorr signature authorises it. Adding one consensus rule, an `OP_SUCCESS` opcode [24] over the byte interface of section 2.3, gives a node mining a spend with no elliptic-curve signature, all 9 mutations tried being refused by it and accepted by an unmodified node — the control attributing the refusals to the new rule. The blocker is a consensus rule, not engineering. Its cost was measured: one verification runs 374 $\mu$s against the curve baseline it is compared with, a BIP340 Schnorr check's 33.0 $\mu$s — 11.3× — and a signature that fails late costs about as much as one that verifies. A patched node is not Bitcoin, so that conclusion stands; the exercise is functional, with no security argument (appendix A.13).

## 3.9 Threats to validity

Several factors qualify these results. All timings are machine-dependent, mitigated by reporting standard deviations, fixing one machine and compiler, emphasising ratios over absolutes, and by the cross-implementation check (section 3.3). The parameter set's concrete security level is not formally established, security proofs being out of scope, so every cross-scheme claim is qualified by table 2.1; the modulus is FIPS 204's $q \approx 2^{23}$ rather than the $2^{24}$ of [9], which leaves correctness unaffected ($q > 2\gamma$) but changes the security margin. The extracted witness is

exact here, whereas the relaxed setting of [9] allows witness noise that can grow across long payment paths.

Two further factors are specific to section 3.7. The UTXO ledger is a model rather than a node (appendix A.4): real transaction objects, real serialised sizes and the real leak channel atomicity depends on, but no fees or confirmation latency, so on-chain byte counts are the honest proxy. That limit belongs to the venue rather than the study: configurations 2 and 3 need a consensus change, built and run in appendix A.13. Finally, the Stage-2 timings come from a single desktop machine (AMD Ryzen 7 7745HX with Radeon Graphics), so the usability discussion is an extrapolation, not a measurement on constrained hardware.

# Chapter 4

# Evaluation

This chapter judges the work against the objectives of section 1.3: the evaluation strategy and its alignment with those objectives, each objective against the evidence of chapter 3, then the implementation challenges that shaped the outcome and the limitations that bound it. Judgement of *what the artefact achieves* is made here; section 5.2 steps back to the process.

## 4.1 Evaluation strategy and its justification

The aim was to establish what adaptor functionality *costs*, so each measured difference had to be attributable to one cause. Four decisions follow, each tied to an objective.

**Correctness is established before any cost is reported** (objective 3). A fast but wrong scheme measures nothing, and no formal proof was in scope, so correctness rests on differential evidence: a pinned known-answer digest reproduced byte-for-byte by a second, independently written implementation (section 3.3). Internal self-consistency is not correctness — section 4.3 gives a case where every primitive passed in isolation and the assembled whole was still wrong.

**The primary comparison is controlled, not merely convenient** (objective 4). LAS is measured against its own simplified base at identical algorithm, parameters and primitives, so the difference between the two paths is the adaptor layer and nothing else. Comparing instead against the optimised CRYSTALS-Dilithium reference would confound the adaptor cost with parameter, packing and hint-vector differences; that comparison is therefore retained only as context, and the classical ECDSA adaptor only as a functionality-matched "price of

post-quantum" baseline.

**Every measurement declares its boundary.** Results are reported at two tiers (section 2.6), because an undeclared boundary is the commonest way a benchmark misleads — as it did twice here before the tiers were fixed. For the same reason each run gates on a directly counted rejection rate against its closed-form prediction, rather than inferring acceptance from timing.

**Metrics follow the venue** (objective 5). The application study reports execution time and communication bytes with off-chain messages counted, because the evaluated UTXO ledger has no gas metric — and because off-chain messaging is where the post-quantum cost lands.

## 4.2 Achievement of the objectives

Table 4.1 sets out the evidence behind each objective. All five were met; two carry qualifications that matter.

Objective 4's fairness claim rests on control rather than assertion: in Stage 1 the two compared paths share algorithm, parameters and primitives, and in Stage 2 the two LAS configurations share one public matrix and receive byte-identical inputs, verified at run time. The measurements are then defended from two independent directions — a closed-form gate on the rejection rate, and re-measurement under a different compiler and an independent statistical harness.

Objective 5 was met, but only after the first venue failed. On a local EVM a *complete*, validated native LAS verifier was measured at ≈56.6 million gas (table 3.7), above the EIP-7825 per-transaction cap, and an optimistic variant then made honest settlement cheap while leaving a worst-case fraud proof that also exceeds it. That negative result is quantified rather than assumed, and it motivated rebuilding the protocol over a UTXO ledger of the kind [9] assumes. That ceiling later proved to bound the *implementation* rather than the scheme: a claim transaction running a restructured verifier was mined inside the cap, at the target parameter set and message length (section 3.6). The venue decision therefore rests on cost, not impossibility.

Table 4.1: Each objective of section 1.3 against the evidence reported in chapter 3. All five were met; the evidence column states what each rests on.

| **Objective** | **Evidence** |
|---|---|
| 1. Literature and scheme selection | Adaptor signature selected as the exotic scheme of highest blockchain value; LAS [9] selected as the design with the shortest path from a standardised lattice signature (section 1.2). |
| 2. Additive implementation | Lattice core reused byte-for-byte, zero upstream functions modified, made auditable by a two-branch diff; adaptor layer, wire encoding and application are new code (table A.7). |
| 3. Validation | Functional, serialization, tamper and known-answer tests pass (section 3.1); an independent Rust implementation reproduces the wire format and the pinned digest byte-for-byte (section 3.3). |
| 4. Fair benchmarking | Adaptor overhead isolated per operation at two declared tiers (table 3.2); two further baselines, computation separated from communication, rejection rate gated against theory, relative conclusions reproduced by the Rust port (table 3.4). |
| 5. Atomic-swap demonstration | End-to-end two-party, two-chain swap settling both legs and asserting unspendability before adaptation (section 3.6); extended to a measured three-configuration study on a UTXO ledger (section 3.7). |

## 4.3 Implementation challenges

Turning the simplified-Dilithium base into LAS was conceptually small — four added operations and one extra term in a hash — but the six problems of table A.1 consumed most of the engineering effort. Three themes recur. Two are failures that *pass every local test*: a loosened PreSign bound leaves the adaptor operations working while breaking ordinary verification two steps later, and a doubly-transformed public matrix produces a verifier whose parts agree with each other but not with the C implementation — both caught only by checking outside the component, which is why section 4.1 treats internal self-consistency as insufficient. Two more are measurement artefacts that would have biased the headline result had they survived, and are why every boundary is now declared and every run gated. The last is asymptotic rather than arithmetic, and generalises past this project: in constraint-system libraries the ergonomic interface and the

scalable interface are not the same interface.

## 4.4 Limitations

The validity threats of section 3.9 bound what the results *mean*. Two further limitations belong to the artefact rather than the measurement. The hint-free construction yields larger keys and signatures than optimised Dilithium — cryptographic optimisation given up for a transparent, testable artefact — and section 3.5 quantifies that price and shows it recoverable in principle, though only for the signature and not the statement. Configuration 2 instead gives up post-quantum assurance for a controlled comparison: it is the intermediate point that makes the proof-system comparison possible, not a stack anyone should deploy, since a post-quantum signature does not protect a swap whose fairness rests on a pairing assumption.